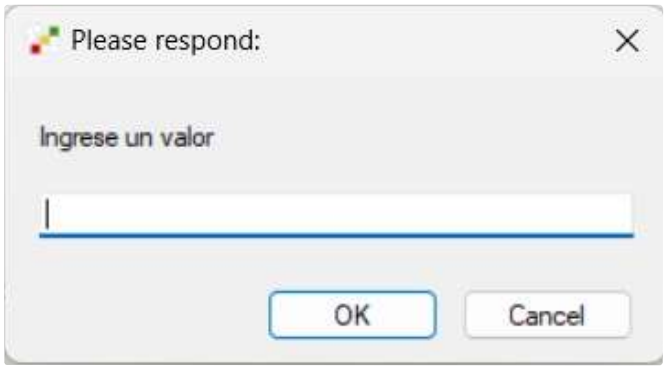
Principales métodos, estructuras y utilidades del Workspace 👍

▼ Prompter 🧙‍♂️

El Prompter es una clase que permite al usuario poder ingresar un valor por pantalla, se expresa de la siguiente manera.

```
Prompter prompt: 'Ingrese un valor'.
```

Una vez ejecutado ese comando, se ve de la siguiente manera



Una vez comprendido el funcionamiento del **Prompter**, es necesario definir un mecanismo para almacenar el valor ingresado por el usuario. Una alternativa es asignar dicho valor a una variable, lo cual puede realizarse de la siguiente manera:

```
var:=Prompter prompt: 'Ingrese un valor'.
```

En este caso “var” va a guardar el valor que se ingrese por el Prompter

¿Que pasa si queremos guardar un valor numérico?

En este caso, es necesario realizar una conversión utilizando el método `asNumber`, que permite transformar la cadena de tipo string en un número.

Se implementa de la siguiente manera

```
var:= (Prompter promt: 'Ingrese un valor')asNumber.
```

Transcript

El Transcript es un clase que permite enviarle mensajes a la ventana System Transcript (Explicada en Paradigmas). Existen distintos métodos de la clase Transcript, nosotros en la cursada vamos a ver los siguientes.

show: Este método te permite mostrar un texto dentro del System Transcript

```
Transcript show: 'Hola mundo'.
```

clear: Este método te permite borrar todo el contenido dentro del System Transcript

```
Transcript clear.
```

cr: Este método te permite realizar un salto de linea dentro del System Transcript

```
Transcript cr.
```

Conversión de datos

Dependiendo del contexto, podemos llegar a necesitar convertir de un tipo de dato a otro. Cuando necesitamos convertir una cadena de caracteres a un número para imprimirlo en el Transcript, lo podemos hacer con el método `displayString.`

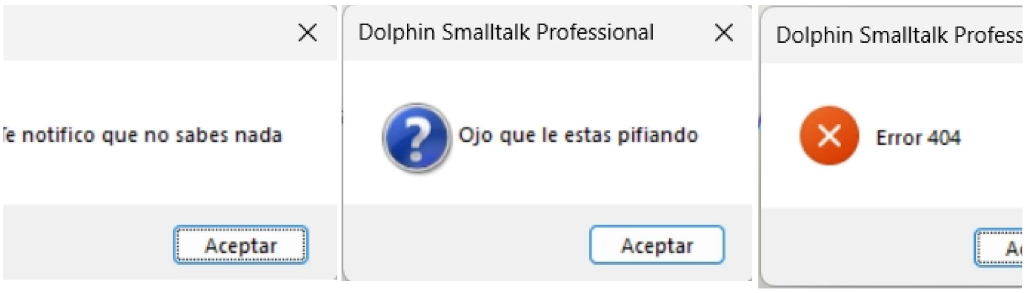
```
Trasncript show: (remedio verPrecio)displayString.
```

MessageBox 📦

El **MessageBox** es una clase que permite mostrar mensajes mediante una ventana emergente. Existen dos tipos:

- Sin valor de retorno: `notify`, `warning` y `error`. Su objetivo es solo informar al usuario, mostrando un mensaje con distinta estética según el tipo de aviso.

```
MessageBox notify: 'Te notifico que no sabes nada'.
MessageBox warning: 'Ojo que le estas pifiando'.
MessageBox errorMsg: 'Error 404'.
```



- Con valor de retorno: `confirm` A diferencia de los anteriores, devuelve un valor booleano (`true` o `false`) según la respuesta del usuario, lo que permite usarlo dentro de funciones o tomar decisiones en el programa.

```
MessageBox confirm: '¿Queres ingresar otro alumno?'
```

Como guardar el valor booleano en una variable

```
rta:= MessageBox confirm: '¿Queres ingresar otro alumno?'
```



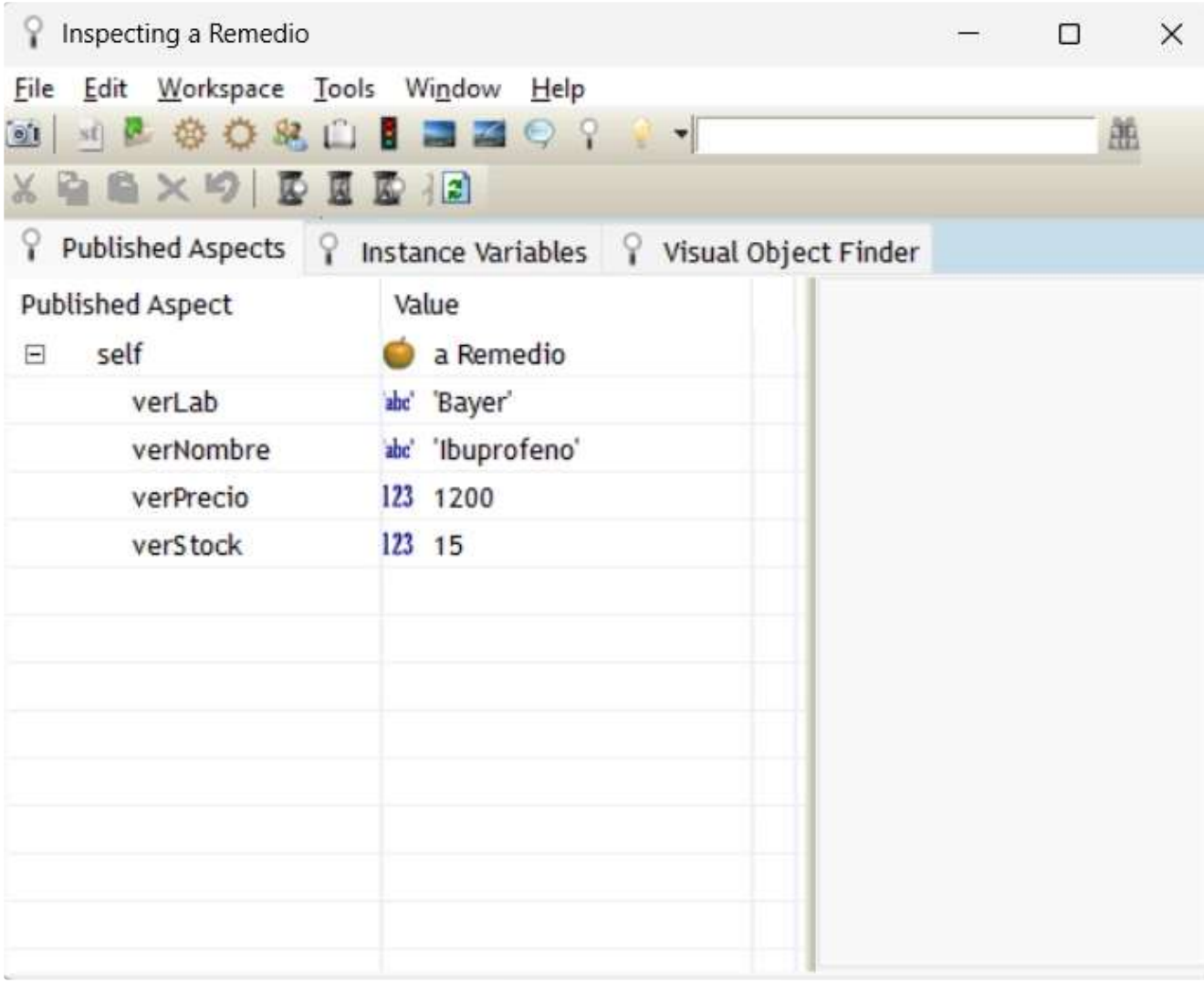
Les dejo una pregunta, que pasa si dentro de la condición del while pongo
"MessageBox confirm: '¿Queres cargar otro alumno?'"
Algo interesante puede hacer, seguro me ahorra un paso y una variable no se 🤔

Inspect

Esta herramienta nos permite a los programadores poder ver que esta pasando dentro de un objeto en tiempo real, ojo a la aclaración, esto es para el programador y **NO para el usuario**
Ejemplo
A nuestra clase remedio le cargamos un nombre, precio, stock y laboratorio como se muestra abajo

```
r1 := Remedio crear: 'Ibuprofeno' con: '1200' asNumber con: '15' asNumber con: 'Bayer'.
r1 inspect.
```

Luego de realizar el inspect, se visualiza la siguiente pantalla donde muestra todos los datos cargados.



En primera linea se visualiza el self que retorna un a Remedio en nuestro caso es la Clase Remedio
Luego todos los atributos con sus respectivos valores
Si tiene un 'abc' quiere decir que guardo un dato de tipo char
Si tiene un 123 quiere decir que guardo un dato de tipo numerico (por el asNumber)

▼ Estructuras de decisión ⚽

En **Dolphin Smalltalk**, las estructuras de decisión se basan en **mensajes enviados a valores booleanos (true o false)** y se utilizan **bloques de código** para definir qué hacer en cada caso.

(condición) `ifTrue:` [bloque de código].
(condición) `ifFalse:` [bloque de código].
(condición) `ifTrue:` [bloque de código] `ifFalse:` [bloque de código].

Nota: En caso de estructura de decisión la condición va entre paréntesis "()"

▼ Estructuras de repetición 🧉

En **Dolphin Smalltalk**, los bucles también se basan en **mensajes enviados a booleanos**, y los más comunes son **whileTrue:** y **whileFalse:**.
[condición] `whileTrue:` [bloque de código].
[condición] `whileFalse:` [bloque de código].

Nota: En caso de estructura de repetición la condición va entre corchetes "[ ]"

▼ Métodos de clase e instancia 🔟

Una vez creada una Clase (explicado en ¿Cómo crear Clases y métodos?) vamos a ver cuáles son sus métodos más comunes dentro del Workspace

▶ Métodos de clase

▶ Métodos de instancia

Iteradores de colección 🥅

Un iterador de colección en Dolphin Smalltalk es un mecanismo que permite recorrer secuencialmente los elementos de una colección.
De esta forma, se puede procesar, filtrar o transformar los datos sin necesidad de manejar índices manualmente.

- do
- reject
- select
- collect
- detect

Menú de opciones 🍔

Una herramienta muy útil para realizar la aplicación es el menú, es una manera mas sencilla e interactiva de poder mostrarle opciones al usuario

Su estructura es muy sencilla, simplemente utiliza un ChoicePrompter choices:

